

Exploration E - Notes

Plotting with ggplot2

In this Exploration, we are going to look at how to construct various types of plots, graphs, and charts.

Structure of a ggplot2 Plot

The first thing you need to do if you want to use plots from `ggplot2` is make sure that the package has been installed and that you have imported the library into your document. I have already installed the package (using the **Install** button in the **Packages** tab of the bottom right pane). But we also need to run the following code to import the library for use in this document.

```
library(ggplot2)
```

Now that we have imported the library, let's explore the basics of a plot made using `ggplot2`.

Grammar of Graphics

Any `ggplot2` plot has an underlying grammar or style that is based on what is known as the *grammar of graphics*. The “grammar of graphics” is the idea that any plot can be built from a series of components: data, a coordinate system, and *geoms* (visual marks that represent data points).

A basic plot in `ggplot2` then usually has at least three components that are joined together using a plus sign `+`. A generic example of the creation of a plot using `ggplot2` looks something like this:

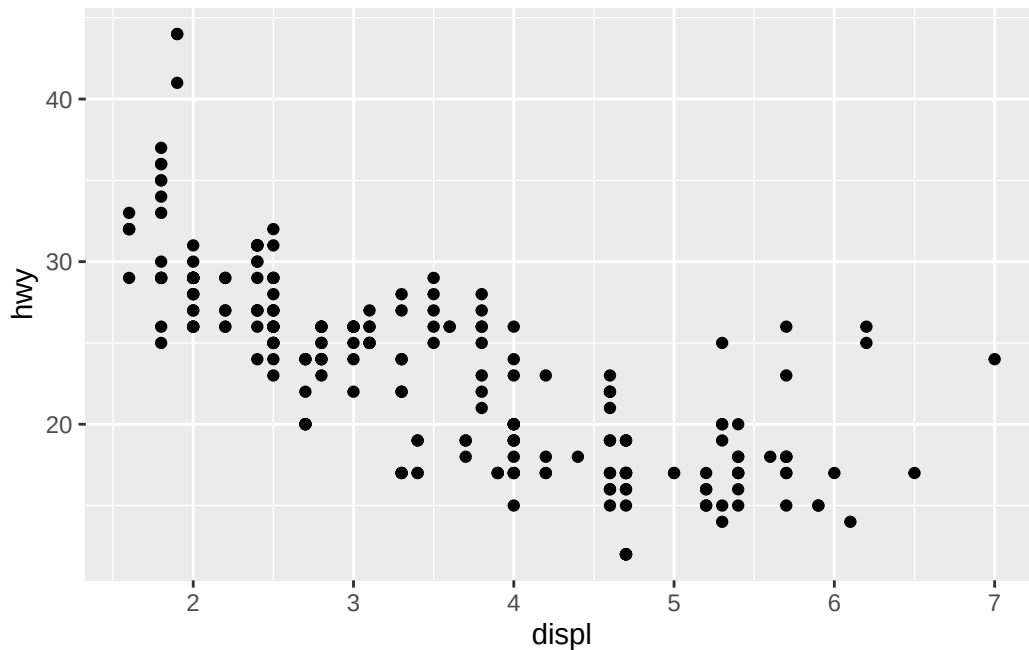
```
ggplot(DF, aes(MAPPINGS)) + GEOM_FUNCTION()
```

where here `DF` would be the name of the data frame from which you are pulling data, `MAPPINGS` would indicate in some way which columns from `DF` will be used to define your plot, and `GEOM_FUNCTION()` will be a name indicating what type of plot you would like to create. Note that `aes` means *aesthetics*, so the information you put in `aes()` is telling R how you want to map data to visual (aesthetic) properties of the plot. Let's see some basic examples.

Scatter Plot + Basic Plot Features

As our first example, let's see how we could use our grammar of graphics to create a basic scatter plot. In this plot we are using the built-in `mpg` data frame, which includes information about cars. On the x-axis will be the engine displacement (`displ`) and on the y-axis will be the highway miles-per-gallon (`hwy`); this is indicated in the code via the aesthetics: `aes(x=displ, y=hwy)`.

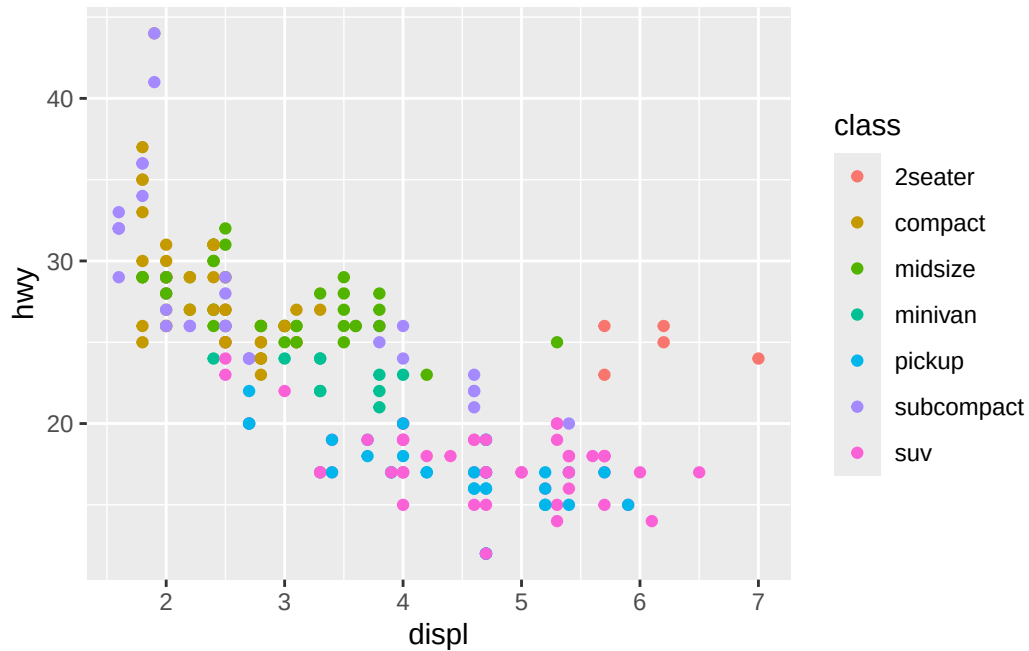
```
# basic scatter plot
ggplot(mpg, aes(x=displ, y=hwy)) +
  geom_point()
```



If we wanted the plot to be a little more interesting, we could easily adjust the color of the dots using the aesthetics. In particular, in the plot below, each dot on the graph (which represents

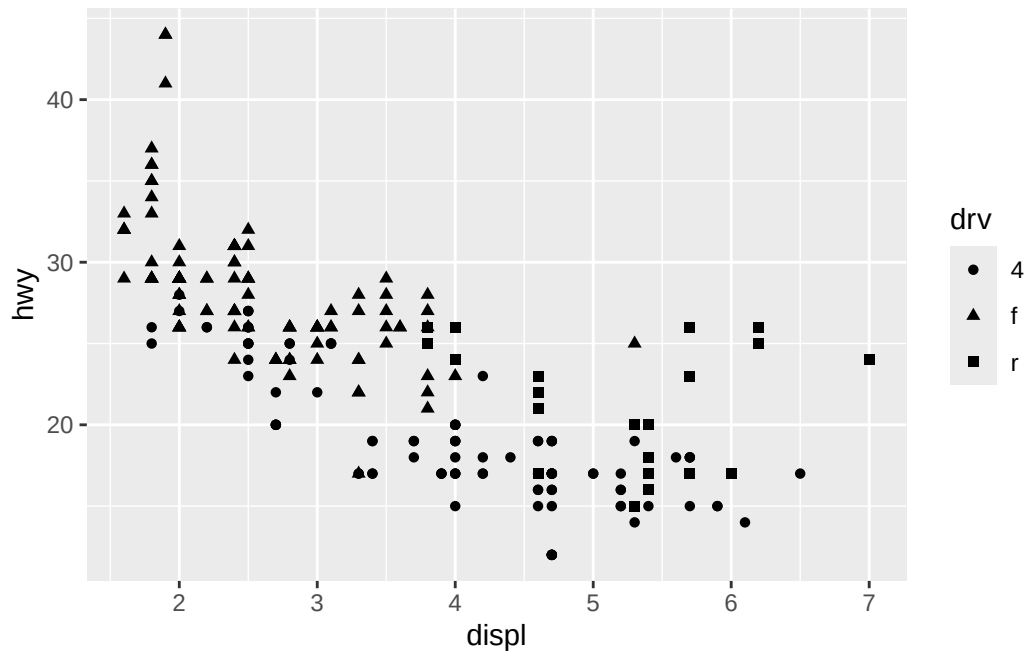
a single car), is colored based on a third (categorical) variable `class`. You'll notice that the plot automatically creates a legend for you.

```
# comparing displ to hwy, by class
ggplot(mpg, aes(x=displ, y=hwy, color = class)) +
  geom_point()
```



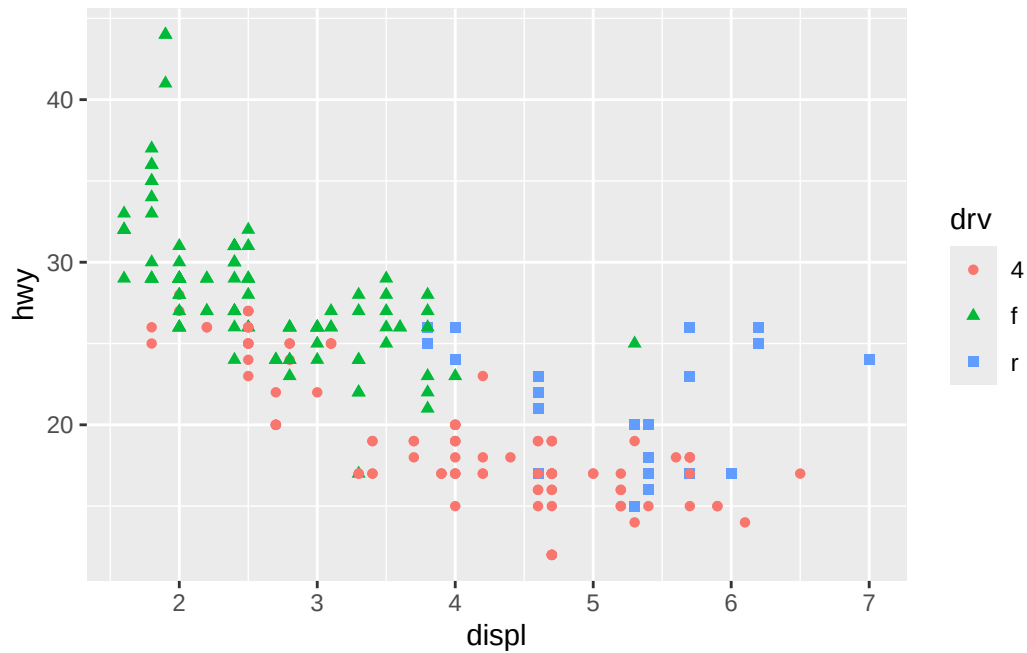
You can change other visual features as well, such as the shape of the points on the scatter plot. In the example below, each dot is a different shape based on the categorical variable `drv`.

```
# comparing displ to hwy, by drv
ggplot(mpg, aes(x=displ, y=hwy, shape=drv)) +
  geom_point()
```



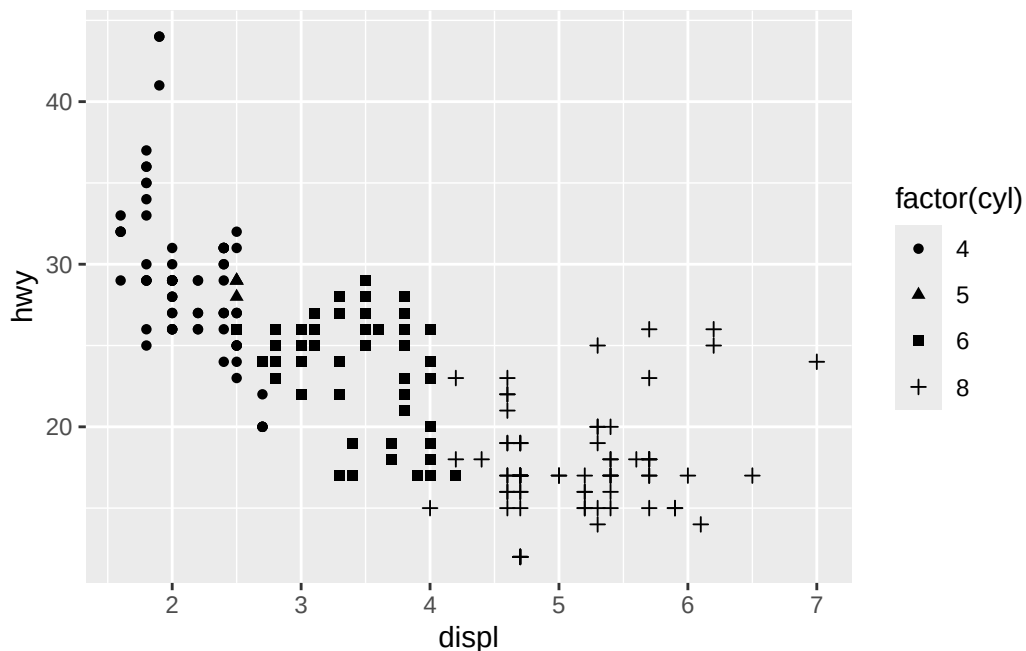
You can even easily change both the shape and the color based on one (or multiple) variable(s). Below the shape and color are both changed depending on `drv`.

```
# comparing displ to hwy, by drv
ggplot(mpg, aes(x=displ, y=hwy, shape=drv, color=drv)) +
  geom_point()
```



Note that if you want to change the shape of your dots based on a *numeric* variable, you may need to `factor()` the variable first, as in this example below.

```
# factoring numeric variable cyl before grouping
ggplot(mpg, aes(x=displ, y=hwy, shape=factor(cyl))) +
  geom_point()
```



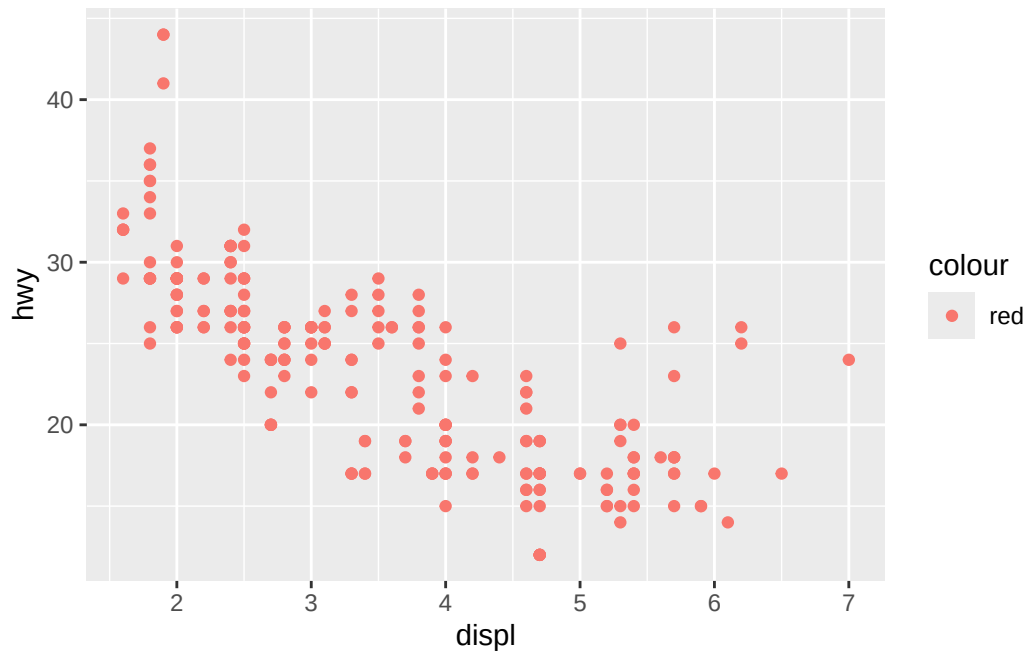
Appropriate usage: scatter plots are used to represent *two quantitative variables*, with one depending on the other (i.e. one an *explanatory variable* and one a *response variable*). As seen above, you can also group this data via a *categorical variable*.

A note on changing aesthetics

If you want to change the aesthetics of the plot without it being based on a particular variable, this can be done but should be done *outside the `aes()` function*. Below you will see the wrong and right way to change the color of all the dots on a plot (without being based on another variable).

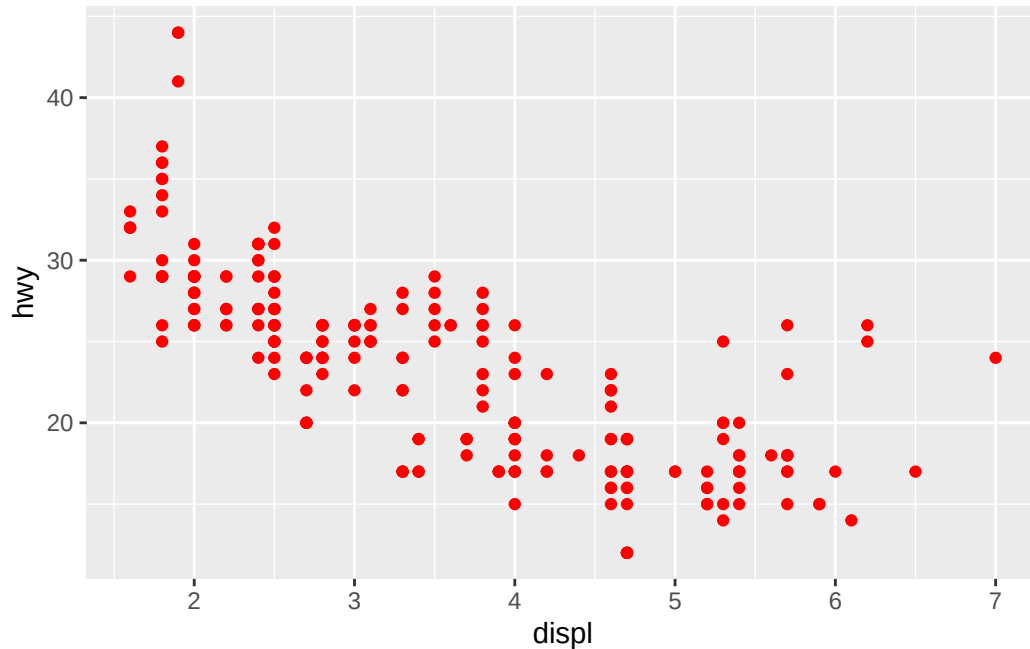
First, the **WRONG WAY**: inside `aes()`, which causes an unneeded legend to be added to the plot.

```
# WRONG WAY: in aes()
ggplot(mpg, aes(x=displ, y=hwy, color="red")) +
  geom_point()
```



The correct method is below. **Notice that the color is changed outside of `aes()` so no unneeded legend is created.**

```
# correct color change
ggplot(mpg, aes(x=displ, y=hwy)) +
  geom_point(color="red")
```

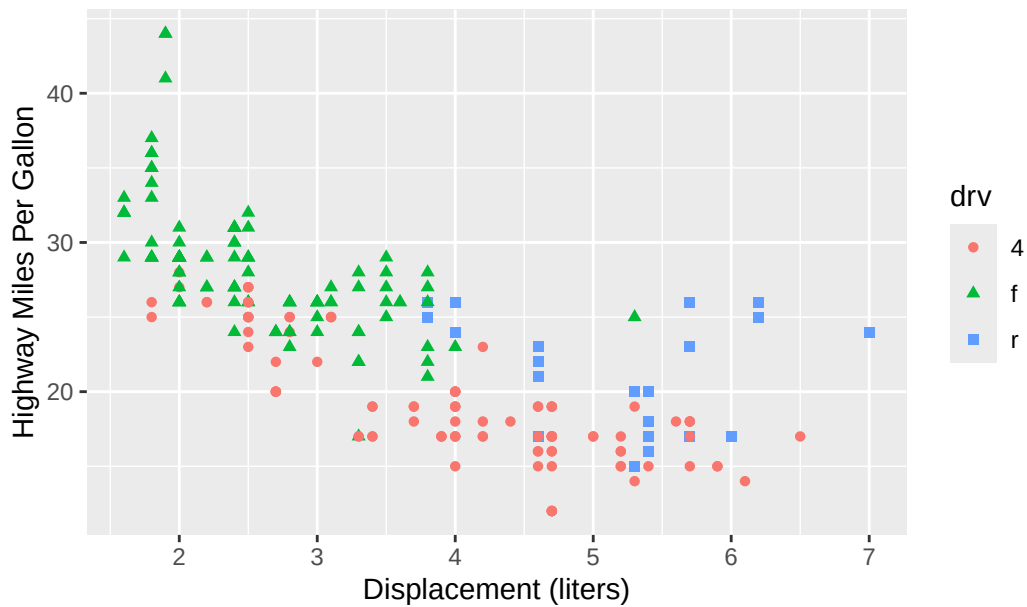


Adding Labels and Titles

Adding labels and titles is quite easy as well and just involves adding onto the grammar of our plot. Below you will see I have added a title to a previous scatter plot, along with new names for the x- and y-axes by adding the component `labs()` to our grammar of graphics.

```
# adding labels and title to plot
ggplot(mpg, aes(x=displ, y=hwy, shape=drv, color=drv)) +
  geom_point() +
  labs(x="Displacement (liters)",
       y="Highway Miles Per Gallon",
       title="Comparing Displacement to Highway MPG")
```

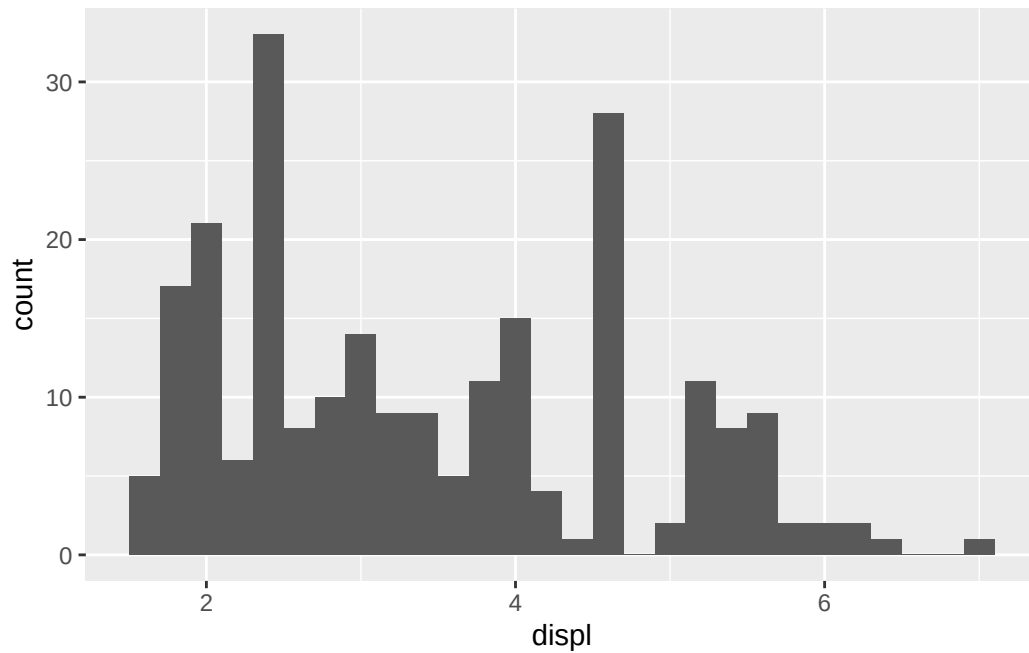
Comparing Displacement to Highway MPG



Histogram

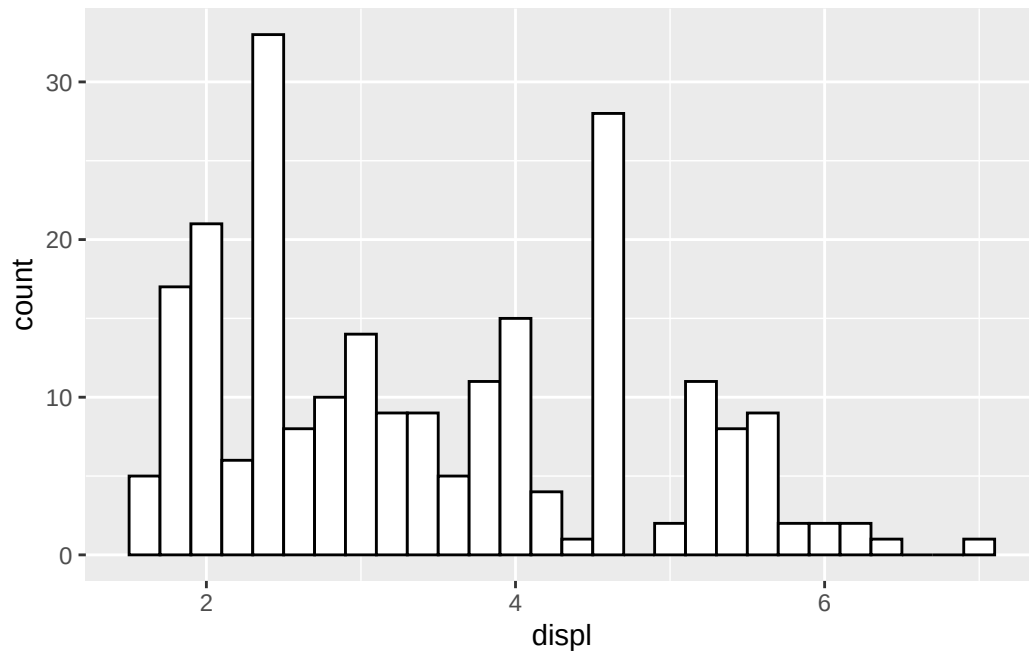
Now that we have seen the basics of creating a plot, let's see some other examples of plots that can be made with `ggplot2`. First: histograms. Here is a basic example (plotting the frequency of the engine displacements). Notice that we use the `geom_histogram` geom for this plot, and additionally that we should include the `binwidth` attribute to indicate the range of values being represented by a single bar (i.e. how wide each bar is).

```
# generic histogram
ggplot(mpg, aes(x=displ)) +
  geom_histogram(binwidth=0.2)
```



We can easily change the colors of the bars or the bar outlines using various attributes, as demonstrated below. Notice that `color` changes the outline and `fill` changes the color of the bars.

```
# histogram with different colors
ggplot(mpg, aes(x=displ)) +
  geom_histogram(binwidth=0.2, color="black", fill="white")
```

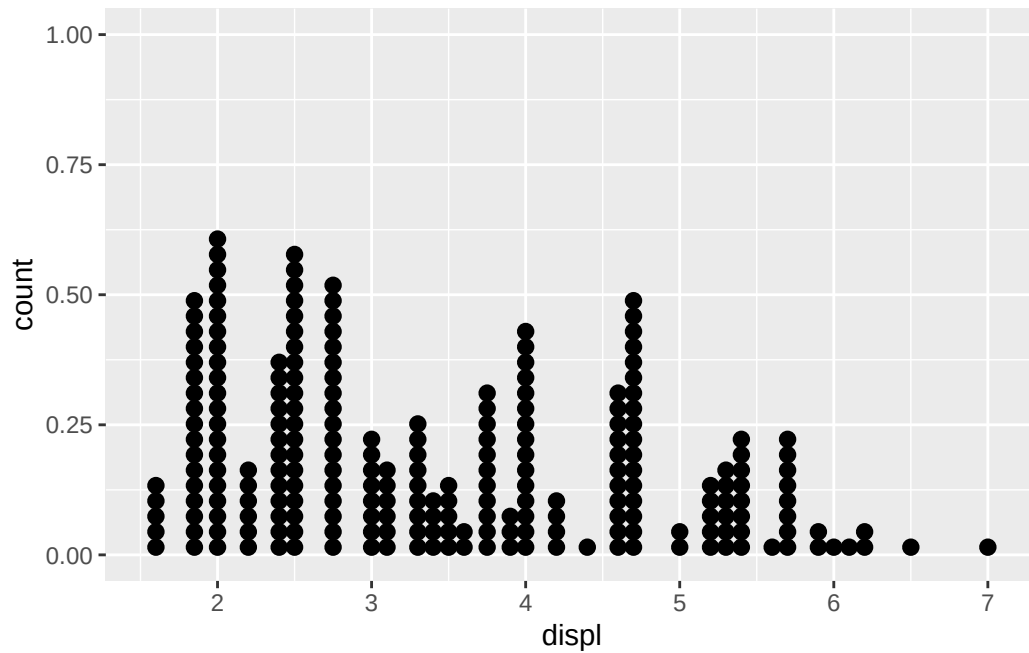


Appropriate usage: A histogram is used to represent frequencies of a *quantitative variable*.

Dot plot

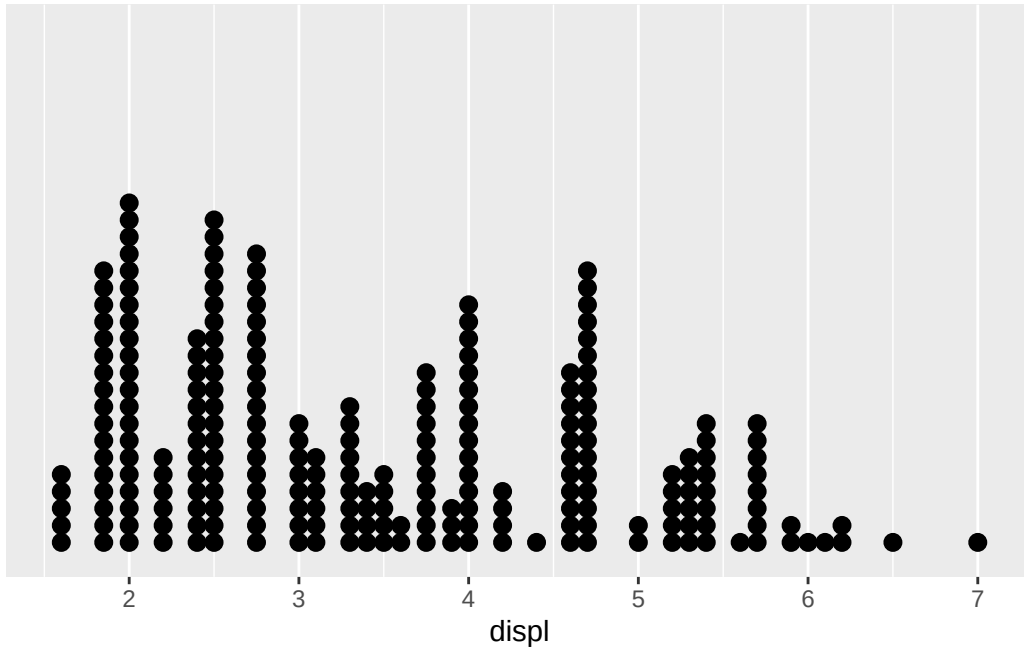
A dot plot is similar to a histogram in that it also displays the frequency of a particular quantitative variable. Here is a dot plot also displaying the frequency of the engine displacement.

```
# generic dot plot
ggplot(mpg, aes(x=displ)) +
  geom_dotplot(binwidth=0.1)
```



Often with a dot plot, the numbers labeling the y-axis are not necessary, so you may consider hiding them by adding the following component.

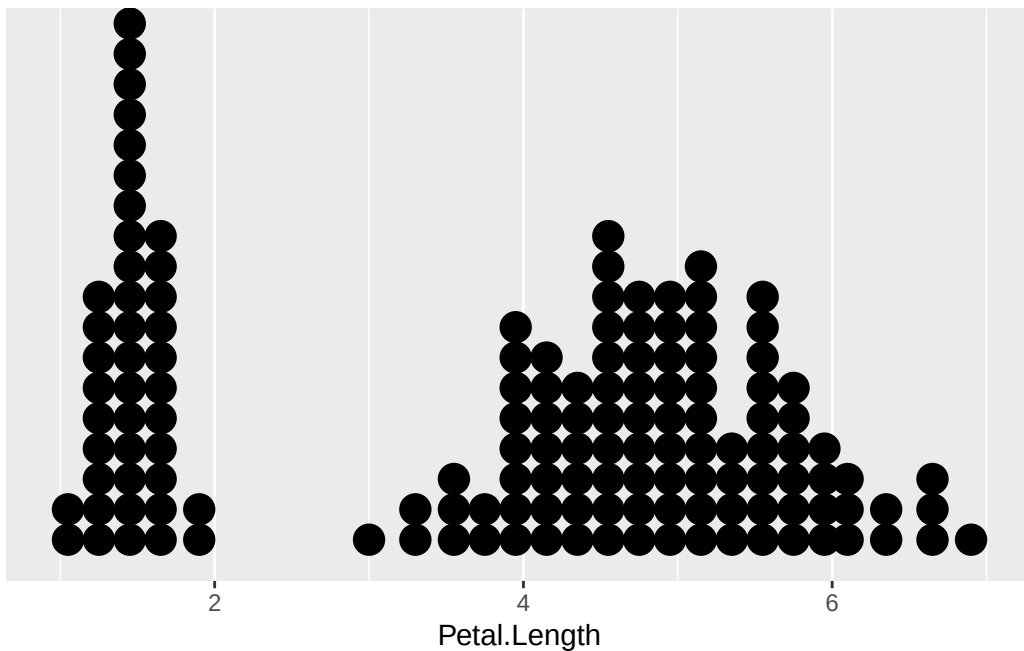
```
# dot plot with no y-axis labels
ggplot(mpg, aes(x=displ)) +
  geom_dotplot(binwidth=0.1) +
  scale_y_continuous(NULL, breaks = NULL)
```



Dot plots in ggplot can be somewhat fickle and sometimes the dots will go off the top of the plot (see example below).

```
# boxplot with some dots off plot
ggplot(iris, aes(x=Petal.Length)) +
  geom_dotplot() +
  scale_y_continuous(NULL, breaks = NULL)
```

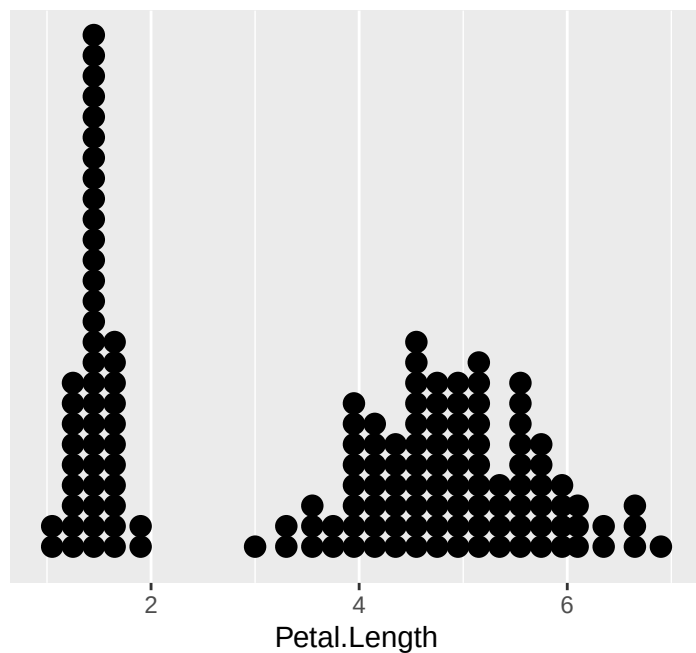
Bin width defaults to 1/30 of the range of the data. Pick better value with ``binwidth``.



One way to fix this problem is to use the `coord_fixed` feature, as in the example below. The `ratio` attribute will adjust how many of the dots are visible. You might consider playing around with this attribute until all the dots are visible.

```
# adjusting so all dots are shown
ggplot(iris, aes(x=Petal.Length)) +
  geom_dotplot() +
  scale_y_continuous(NULL, breaks = NULL) +
  coord_fixed(ratio=5)
```

Bin width defaults to 1/30 of the range of the data. Pick better value with ``binwidth``.

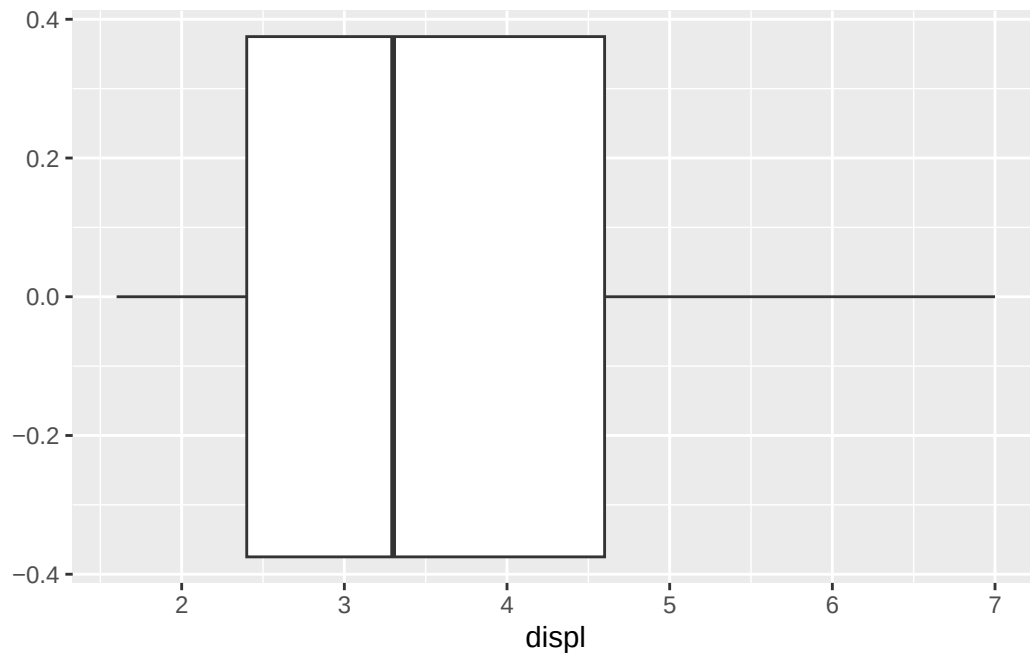


Appropriate usage: A dot plot is used to represent exact frequencies of a *quantitative variable*.

Boxplot

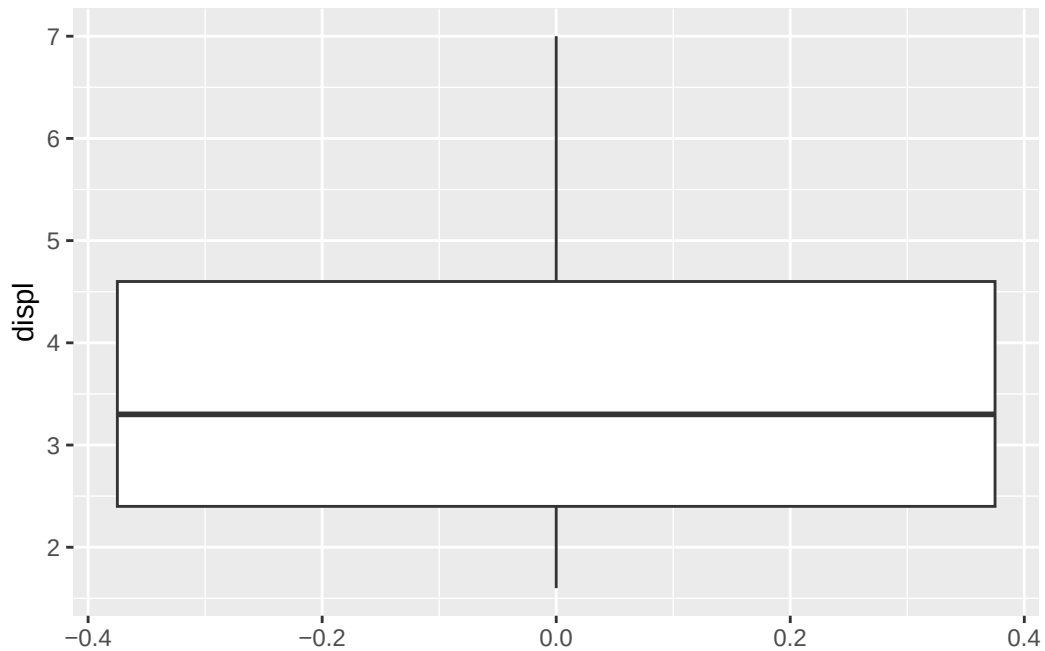
Boxplots are also useful for displaying quantitative data. Here is an example plotting the frequency of the `displ` variable again.

```
# horizontal boxplot  
ggplot(mpg, aes(x=displ)) +  
  geom_boxplot()
```



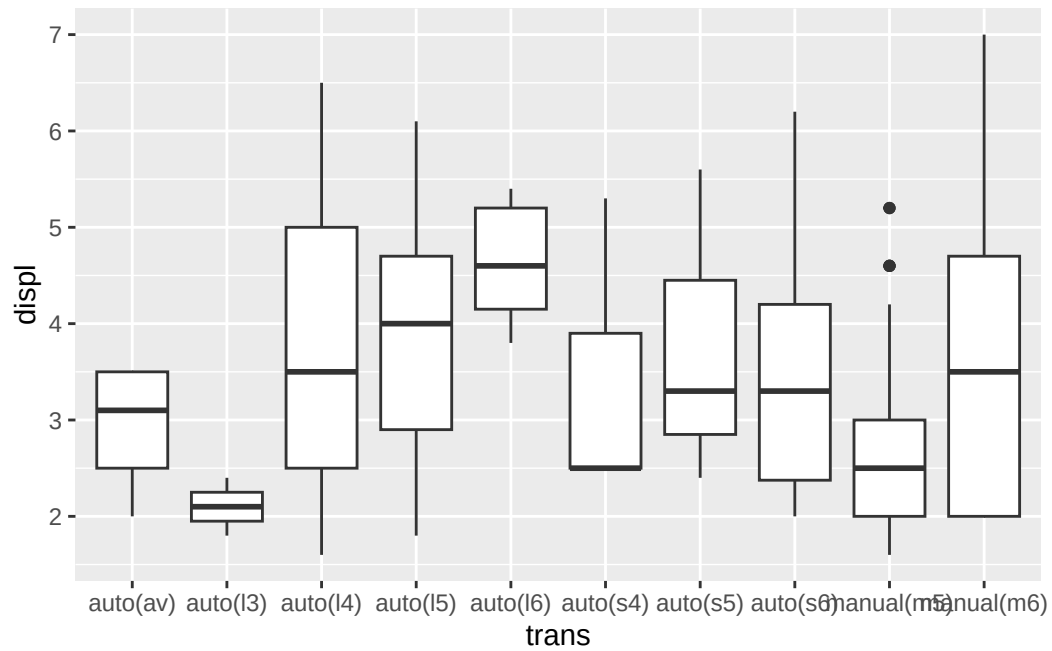
Note that the direction of the boxplot will change depending on whether you use `x` or `y` in the `aes()` mapping.

```
# vertical boxplot  
ggplot(mpg, aes(y=displ)) +  
  geom_boxplot()
```



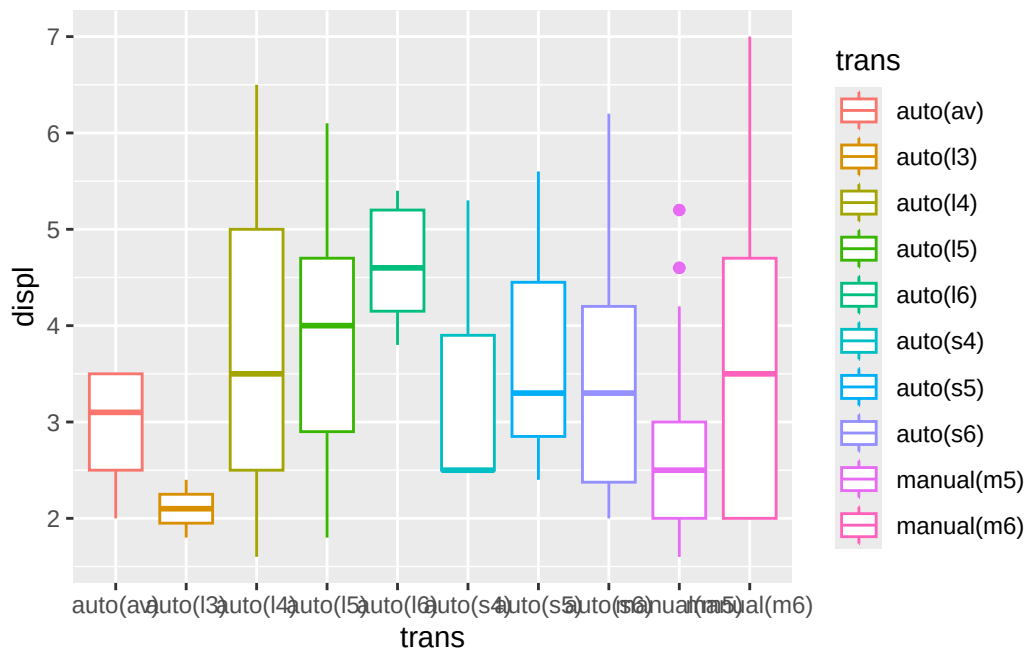
You can also easily create multiple boxplots depending on a particular categorical variable by including a second variable in the `aes()` mapping. Below we include a new boxplot summarizing the `displ` variable for each category of transmission.

```
# multiple boxplots
ggplot(mpg, aes(y=displ, x=trans)) +
  geom_boxplot()
```



As with the scatter plots, you can also easily change the color of the boxes based on the categorical variable.

```
# multiple boxplots with color
ggplot(mpg, aes(y=displ, x=trans, color=trans)) +
  geom_boxplot()
```

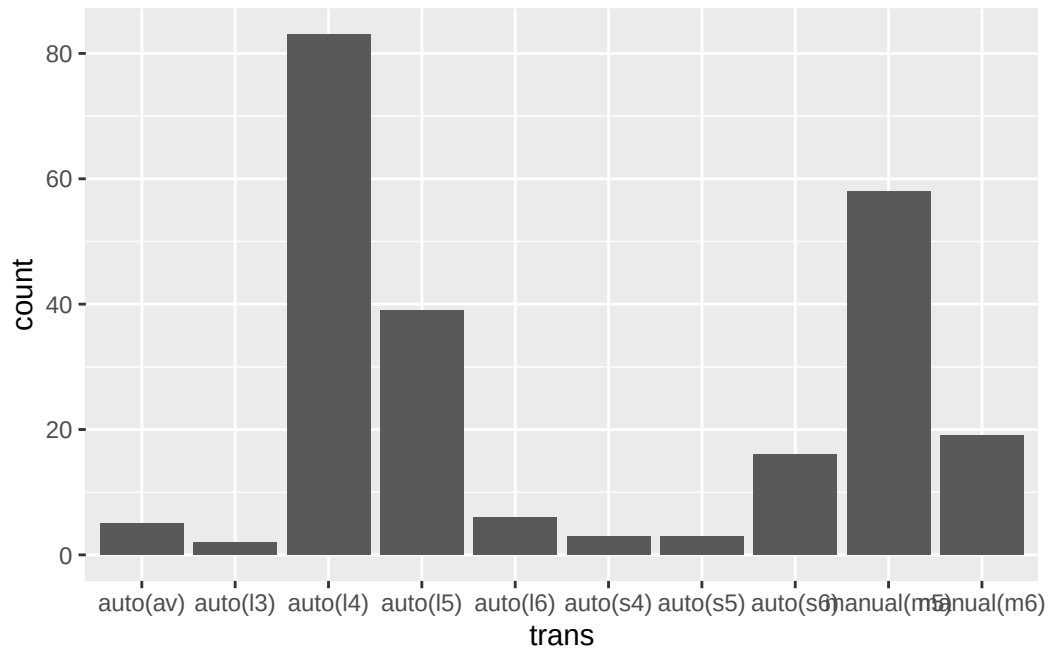


Appropriate usage: a boxplot is used to represent summary statistics (like median, quartiles, IQR) for a *quantitative variable*. As shown above, you can also create multiple boxplots, each representing the same quantitative variable, but grouped via a *categorical variable*.

Bar Graph

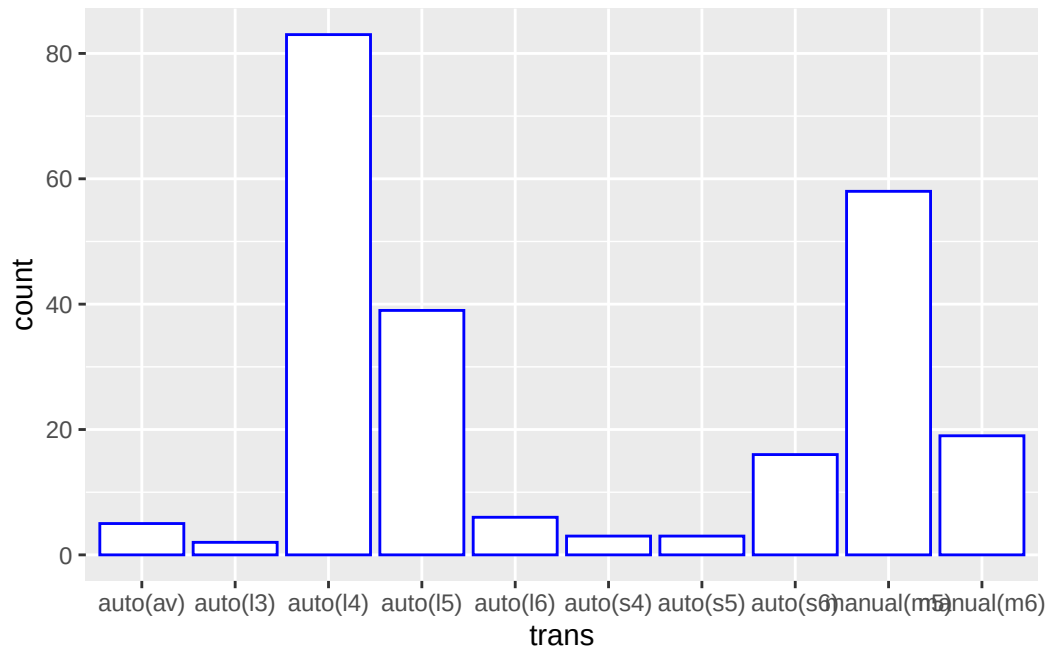
When plotting frequencies of a categorical variable, one plot type you may want to use is a bar graph. Below we plot the frequency of each transmission type.

```
# generic bar graph
ggplot(mpg, aes(x=trans)) +
  geom_bar()
```



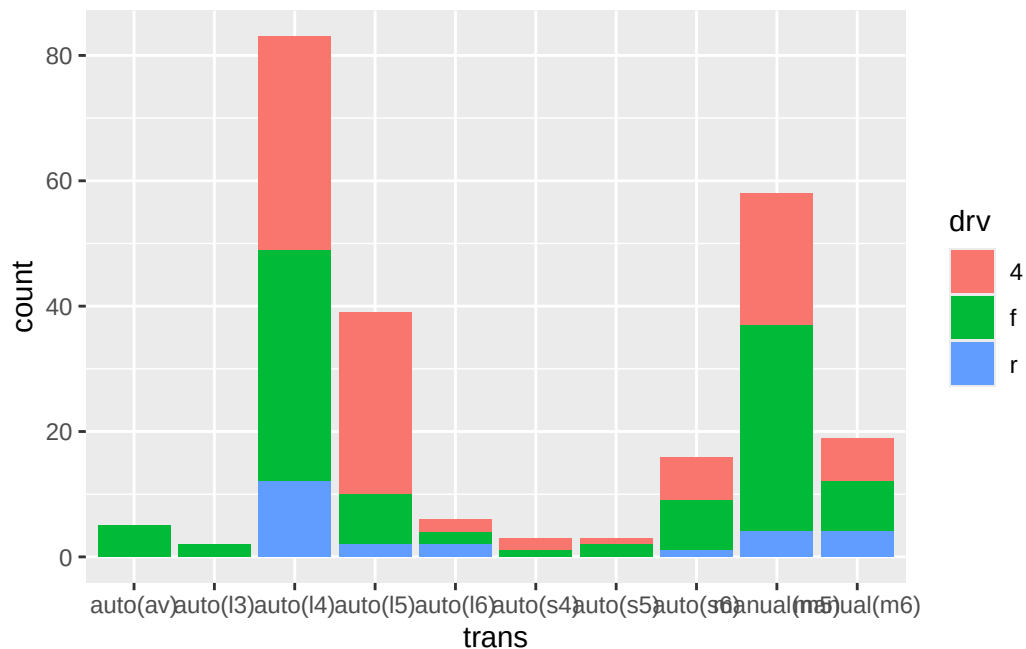
Again, you can use `color` and `fill` to change the colors of the bars. Remember, if you want to change the colors without regard to a particular variable, you will want to use these attributes outside of the `aes()` mapping.

```
# bar graph with different colors
ggplot(mpg, aes(x=trans)) +
  geom_bar(color="blue", fill="white")
```



You could also create segmented bar plots by indicating that the colors should be different based on another categorical variable. Here we create different colored segments based on whether the car has front, rear, or four wheel drive (`drv`) – remember that `fill` (not `color`) changes the color of the bars, and should be placed inside `aes()` since we want the color to be dependent on a variable!

```
ggplot(mpg, aes(x=trans, fill=drv)) +
  geom_bar()
```



Appropriate usage: bar graphs are used to represent *frequencies of categorical variables*. As we saw above, we can also represent frequencies from two-way tables of categorical variables.

Mosaic Plot

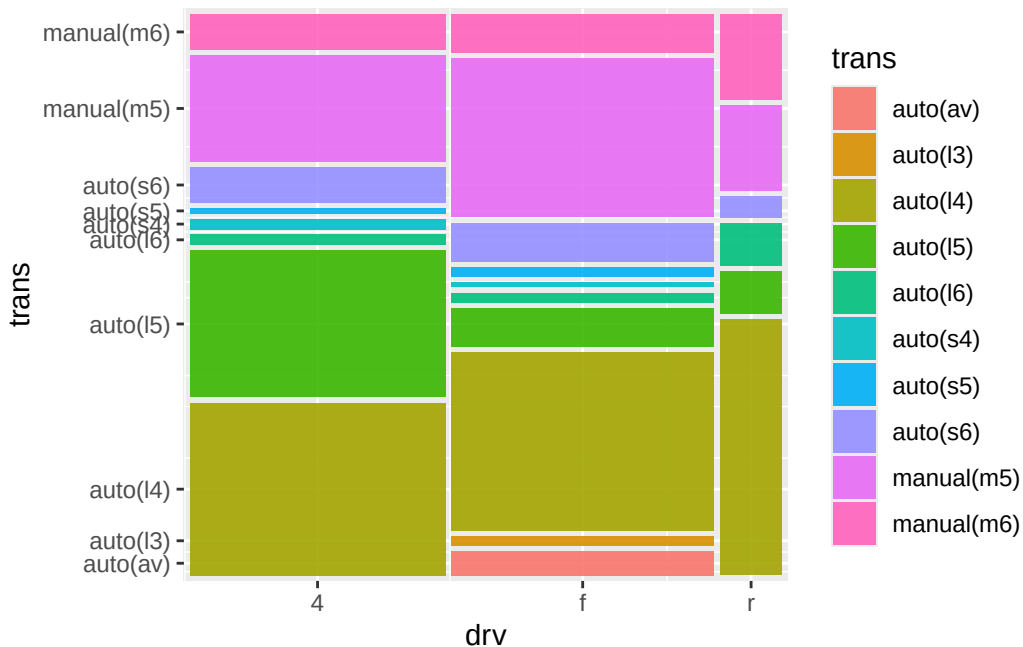
Mosaic plots (sometimes called marimekko plots) are very similar to segmented bar plots, but instead of representing exact frequencies, they represent proportions of each category. They can be created with the inclusion of the `marimekko` package/library. I have already installed the `marimekko` package in this project, but we also need to import the library to this document.

```
library(marimekko)
```

Warning: package 'marimekko' was built under R version 4.5.3

Once the library has been imported, we can now use the `geom_marimekko()` geom to create mosaic plots. Here, we create a mosaic plot comparing the two categorical variables `trans` and `drv`. Note that there is slightly different syntax required for the mosaic plot compared to the rest of the plots we've seen so far.

```
ggplot(mpg) +  
  geom_marimekko(formula = ~ drv | trans)
```



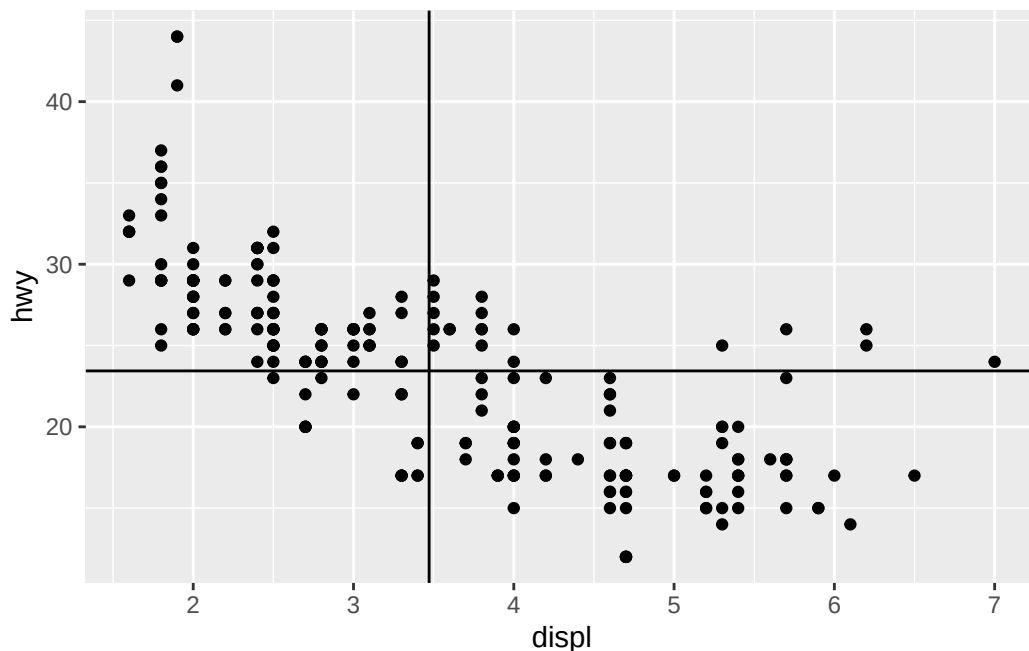
Appropriate usage: mosaic plots are used to represent proportions of *two or more categorical variables*.

Adding Straight Lines

Just as with Base R, it is also possible with ggplot2 to add lines to plots. Vertical or horizontal lines can be included by adding the `geom_vline()` or `geom_hline()` components. More generic lines can be added using the `geom_abline()` component, which allows you to specify the y-intercept and slope (using the $y=ax+b$ form of a line).

In the example below, I have a scatter plot with both a vertical and horizontal line added, each at the mean of the respective variables.

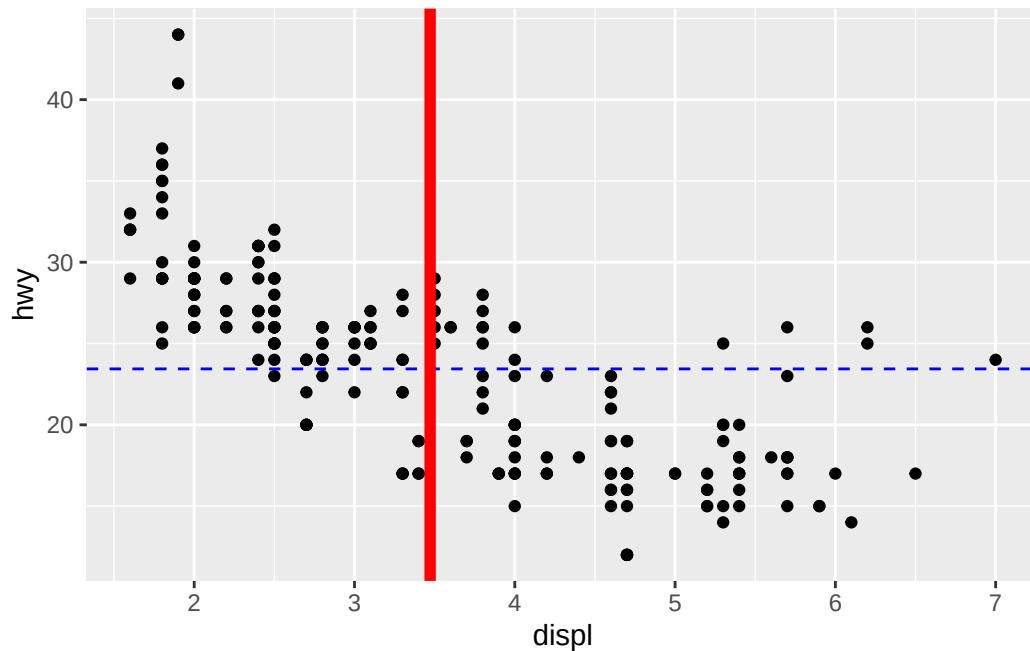
```
# adding two lines
ggplot(mpg, aes(x=displ, y=hwy)) +
  geom_point() +
  geom_hline(aes(yintercept=mean(hwy))) +
  geom_vline(aes(xintercept=mean(displ)))
```



You can adjust the colors, line type (e.g. "solid", "dashed", "dotted"), or size of the line. Notice that the attributes go outside the `aes()` mapping. Notice also that we actually have several instances of `aes()` usage in this plot: in the initial `ggplot()` component (indicating what data to use for the scatter plot), and in each of the line components (indicating what data to use to draw the lines).

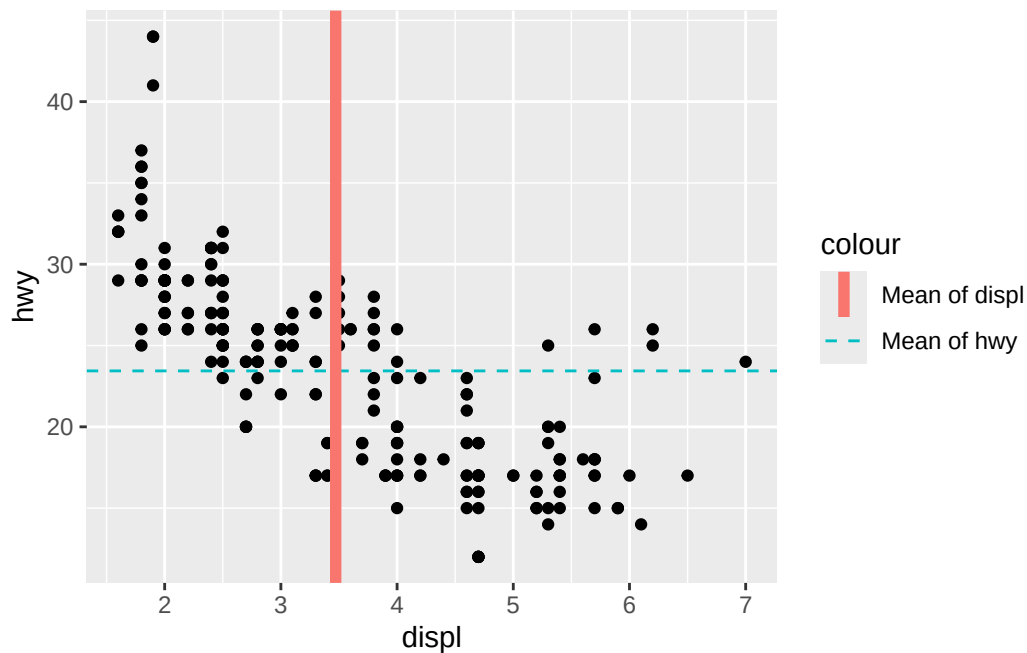
```
# two lines with color and line type adjusted
ggplot(mpg, aes(x=displ, y=hwy)) +
  geom_point() +
  geom_hline(aes(yintercept=mean(hwy)), color="blue", linetype="dashed") +
  geom_vline(aes(xintercept=mean(displ)), size=2, color="red")
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.



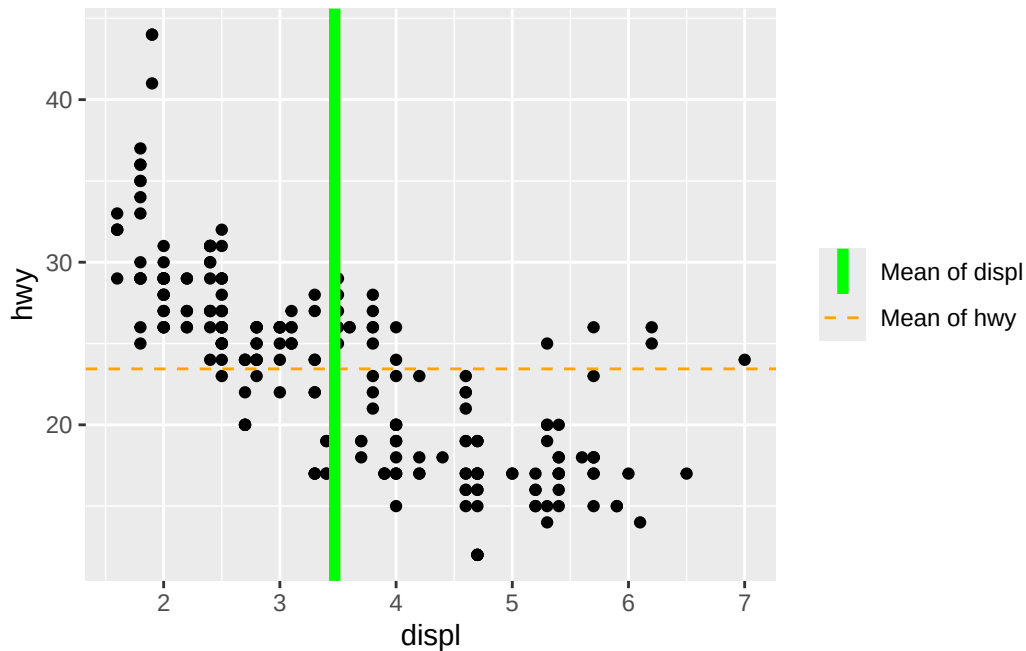
If you want to add a legend to your plot to indicate what the lines mean, you have to do a bit of a funny trick. First: move your `color` attributes inside the `aes()` mappings in the line components and change the color to the name that you want to appear in the legend. This will give default colors to the lines and will add a legend.

```
# lines with color (default), line type, and a legend
ggplot(mpg, aes(x=displ, y=hwy)) +
  geom_point() +
  geom_hline(aes(yintercept=mean(hwy), color="Mean of hwy"), linetype="dashed") +
  geom_vline(aes(xintercept=mean(displ), color="Mean of displ"), size=2)
```



If you want to then change the colors of the lines, you can add one extra component `scale_colour_manual()` indicating what colors you want the lines to be. You can also remove the unneeded “colour” title on your legend using the `labs()` component.

```
# lines with color (custom), line type, and a legend
ggplot(mpg, aes(x=displ, y=hwy)) +
  geom_point() +
  geom_hline(aes(yintercept=mean(hwy), color="Mean of hwy"), linetype="dashed") +
  geom_vline(aes(xintercept=mean(displ), color="Mean of displ"), size=2) +
  scale_colour_manual(values = c("green", "orange")) +
  labs(color=NULL)
```

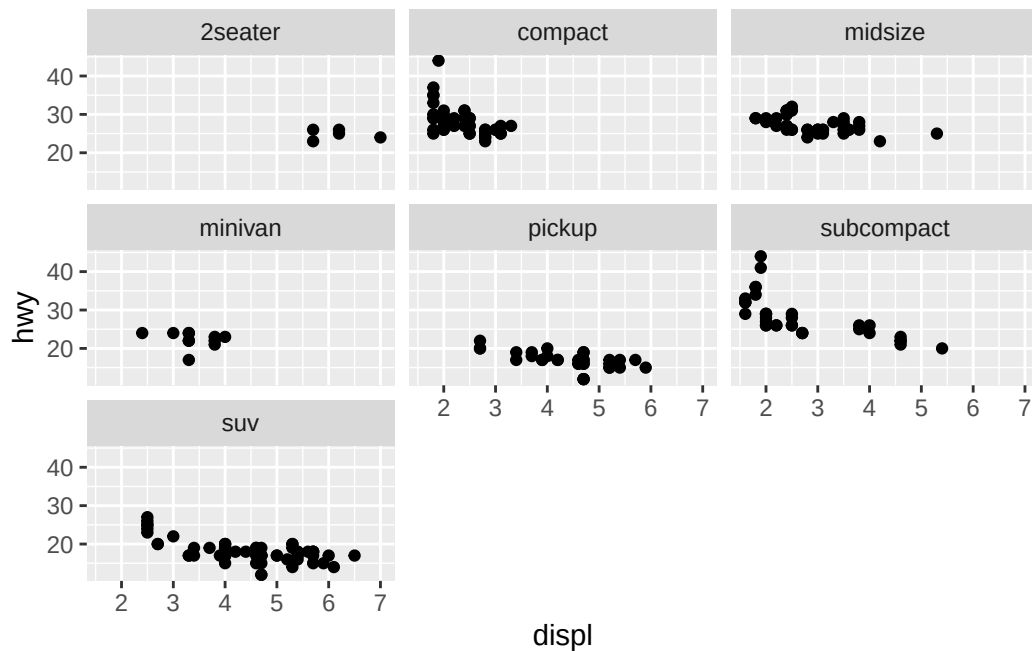


Faceting (Dividing a Plot into Multiple Plots)

The last useful feature I will mention in these notes is the idea of “faceting”. Faceting creates tables of graphics by splitting the data into subsets and displaying the same graph for each subset. To facet, you can use the `facet_wrap()` component, which automatically determine how to organize the data.

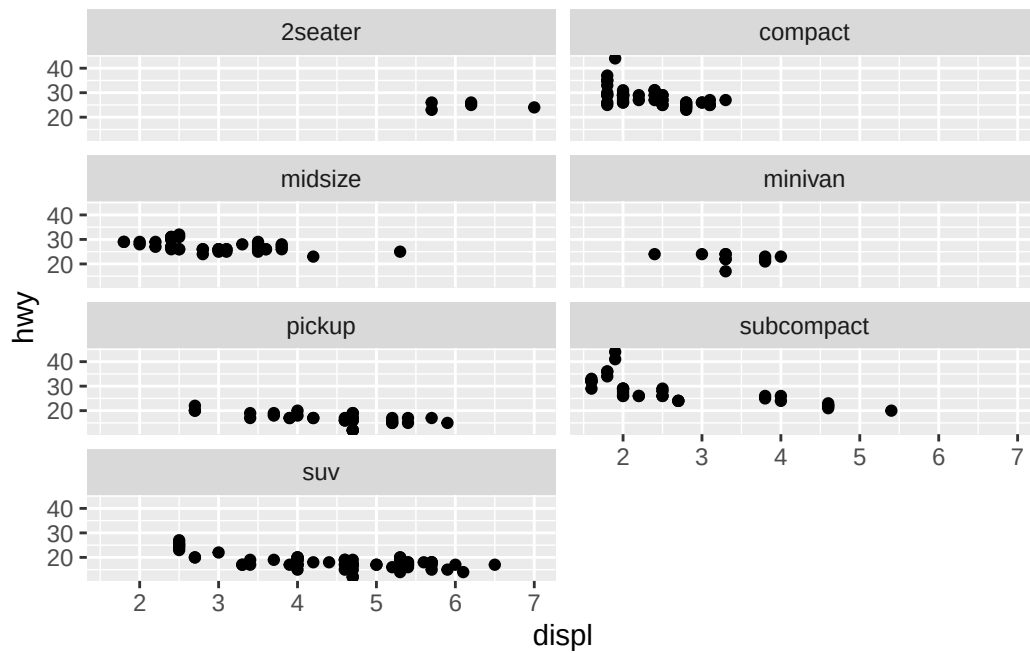
In this first example, we create a grid of plots where each plot compares `displ` and `hwy` for a different `class` of car. Notice that each individual scatter plot is displaying two variables (`displ` and `hwy`) so there are two variables in the `aes()` function, and the third variable (`class`) is being used to divide the data into three scatter plots and is therefore the input to `facet_wrap()`.

```
# generic facet wrap
ggplot(mpg, aes(x=displ, y=hwy)) +
  geom_point() +
  facet_wrap(~class)
```



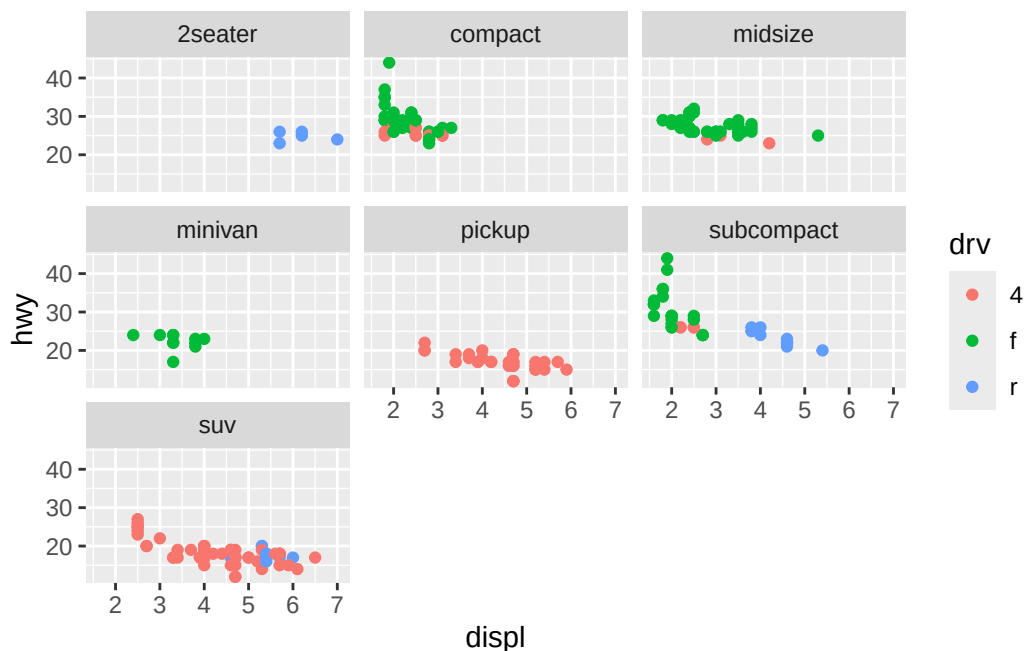
You can specify how many rows or columns you would like using the `nrow` and `ncol` attributes of `facetwrap()`.

```
# facet wrap with custom column number
ggplot(mpg, aes(x=displ, y=hwy)) +
  geom_point() +
  facet_wrap(~class, ncol=2)
```



You can also still change colors based on a particular variable.

```
# facet wrap with color based on drv
ggplot(mpg, aes(x=displ, y=hwy, color=drv)) +
  geom_point() +
  facet_wrap(~class)
```



Adding lines to faceted plots

It can be nice to add lines representing means, medians, or other values to a faceted plot. This requires a little extra work than adding lines to an un-faceted plot, so I have included an example below.

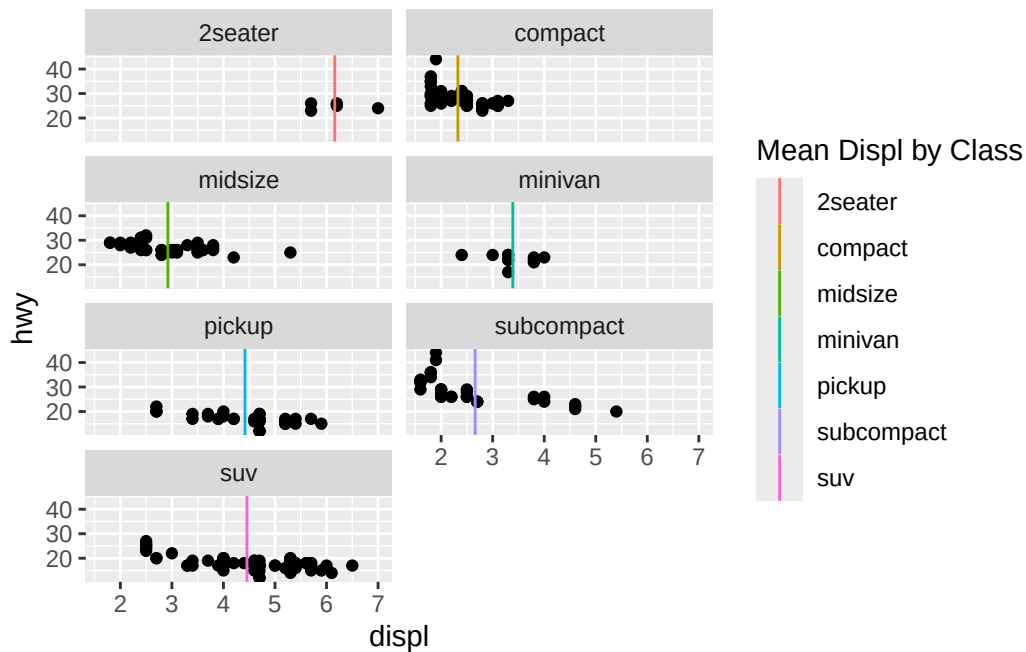
The basic outline is that we create two vectors first: one containing the means that we would like to plot, and one containing the unique values in our categorical variable that we faceted by. We put these into a data frame, and use the information in that data frame to add the lines. Note that the names of the two vectors we create should be the

```
# create two vectors with SAME NAMES as the columns in the data frame they correspond to
displ <- tapply(mpg$displ, mpg$class, mean)
class <- levels(factor(mpg$class))

# create a data frame with those two vectors as columns
means <- data.frame(displ, class)

# the second to last line lets you create the lines
# the last line in the below lets you change the title of the legend
ggplot(mpg, aes(x=displ, y=hwy)) +
  geom_point() +
  facet_wrap(~class, ncol=2) +
```

```
geom_vline(aes(xintercept = displ, color=class), means) +
guides(color = guide_legend(title = "Mean Displ by Class"))
```



What makes a good plot?

It takes practice both to create and identify good plots. In general plots should have the following properties.

- 1) **Informative Title:** A title should contain a concise description of what is being plotted. In general, when thinking of a title consider including: who, what, when, where, why, and how. Depending the context and audience you might not need all of these elements. For instance, if you have a standard scatter plot, you do not necessarily need to list "Scatter Plot". This font should be larger compared to other text on the plot. The title should fit comfortably on the figure.
- 2) **Appropriately Labeled Axis:** Use appropriate axis labels. Instead of simply saying "X" or "Y", say what the variables of these axes are. Include the units of measurement on these axis labels. Use correct capitalization and punctuation. Make sure the font is reasonably sized. A rule of thumb is for it to be about half the size of the title font. Make sure tick marks and unit intervals on an axis are consistent and appropriate for the scale.

- 3) **Nothing Cutoff:** Points, lines, titles, labels, text should all fit comfortably on the plot. There should not be excessive white space, and we should not have values so zoomed in that we cannot see other points. With particularly large data sets we can consider taking a simple random sample, and only plotting those points for ease of readability.
- 4) **Units of measurement listed:** Include units of measurement for the different axes, and for additional graphical parameters as necessary.
- 5) **Legends (when applicable):** Legends should be included if elements of the plot differ by different variables. Legends can be within the plotting area, or outside of the plotting area. Legends can vary greatly depending on the context. Often times the legend font is smaller than else where on the plot.
- 6) **Communicate a Clear Message:** When creating plots it can be tempting to make the plot be too generic or add several elements. Try to have one clear message and objective when creating a plot. Make sure to know your audience. That will also dictate how much you can communicate, and what level of detail needs to be included.

This is not an exhaustive list, but contains a general quick check list of items to consider when creating plots. What constitutes a “bad” plot and a “good” plot can be subjective. Use your best judgment and be creative.

A note on extra resources

There are many more options available to you when making plots using `ggplot2`. There are a few helpful guides (called “cheat sheets”) on Canvas that are worth taking a look at!